

1 Perceptron Rosenblatta

Perceptron to jeden z najstarszych algorytmów uczenia maszynowego, zaproponowany przez Franka Rosenblatta w 1958 roku. Służy do klasyfikacji binarnej i opiera się na uproszczonym modelu neuronu biologicznego. Model ten stanowi ważny krok w rozwoju sieci neuronowych i uczenia maszynowego. Kluczową ideą perceptronu jest obliczenie liniowej kombinacji cech wejściowych i przypisanie ich do jednej z dwóch klas na podstawie funkcji aktywacji — najczęściej funkcji znaku (sign). Jeśli suma ważona cech wejściowych przekracza ustalony próg, próbek zostaje zaklasyfikowana jako należąca do jednej klasy, w przeciwnym razie — do drugiej.

Algorytm uczy się poprzez iteracyjne aktualizowanie wag, tak aby zmniejszać liczbę błędów klasyfikacji na zbiorze treningowym. Działa to w ten sposób, że jeśli aktualna predykcja modelu różni się od rzeczywistej etykiety, to wagi są korygowane w kierunku, który zwiększyłby prawdopodobieństwo poprawnej klasyfikacji tej próbki w przyszłości. Proces ten jest kontynuowany aż do momentu osiągnięcia akceptowalnej liczby błędów lub do wyczerpania maksymalnej liczby epok treningowych. Perceptron jest gwarantowany do znalezienia rozwiązania tylko wtedy, gdy dane są liniowo separowalne, co jest jego głównym ograniczeniem.

Pomimo tej wady, perceptron ma fundamentalne znaczenie w historii rozwoju sztucznej inteligencji. Jest podstawą do budowy bardziej złożonych architektur, takich jak sieci wielowarstwowe (MLP), które potrafią rozwiązywać znacznie bardziej skomplikowane zadania klasyfikacji. Dodatkowo, perceptron wprowadza podstawowe pojęcia, takie jak funkcja aktywacji, wagowanie cech czy proces iteracyjnego uczenia, które są obecne we współczesnych algorytmach uczenia maszynowego.

2 Maszyny wektorów podpierających (SVM)

Maszyny wektorów podpierających (SVM) to jedne z najbardziej popularnych i efektywnych algorytmów klasyfikacji w uczeniu maszynowym. Ich głównym celem jest znalezienie optymalnej hiperpłaszczyzny, która najlepiej rozdziela dane dwóch klas w przestrzeni cech. Optymalność ta oznacza maksymalizację marginesu, czyli odległości między hiperpłaszczyzną a najbliższymi punktami obu klas — tzw. wektorami nośnymi.

W kontekście klasyfikacji liniowej, SVM rozwiązuje problem optymalizacji, w którym minimalizowana jest norma wektora wagowego $\mathbf{w}$, przy jednoczesnym zachowaniu warunku poprawnej klasyfikacji wszystkich punktów treningowych. W praktyce jednak dane często są nieliniowo separowalne. W takich przypadkach stosuje się miękki margines (soft margin), który pozwala na pewne błędy klasyfikacji w zamian za bardziej elastyczne dopasowanie modelu do danych.

Kolejnym istotnym rozszerzeniem SVM jest zastosowanie funkcji jądra (kernel trick), która umożliwia transformację danych do przestrzeni o wyższej wymiarowości, gdzie separacja liniowa staje się możliwa. Popularne funkcje jądra to funkcja radialna (RBF), wielomianowa czy sigmoidalna. Dzięki temu SVM staje się potężnym narzędziem do rozwiązywania problemów klasyfikacyjnych, w których relacje między cechami nie są liniowe.

SVM cechuje się wysoką dokładnością, szczególnie w przypadkach, gdy liczba wymiarów jest znacznie większa niż liczba próbek. Dzięki zastosowaniu jądra i możliwości regulacji marginesu, model ten jest bardzo elastyczny i nadaje się do wielu zastosowań praktycznych, takich jak rozpoznawanie obrazów, klasyfikacja tekstów, analiza bioinformatyczna, a nawet predykcja finansowa.

3 Klasyfikatory oparte na regule Bayesa

Klasyfikatory oparte na regule Bayesa wykorzystują probabilistyczne podejście do przypisywania obiektów do klas. Kluczowym elementem tej metody jest twierdzenie Bayesa, które pozwala wyznaczyć prawdopodobieństwo przynależności obiektu do danej klasy na podstawie rozkładów warunkowych i apriorycznych. Model ten przyjmuje, że dla każdej klasy można określić prawdopodobieństwo wystąpienia konkretnego wektora cech, a klasyfikacja polega na wyborze klasy z najwyższym prawdopodobieństwem a posteriori.

Reguła Bayesa w praktyce umożliwia wykorzystanie informacji statystycznej do podejmowania decyzji klasyfikacyjnych nawet przy niepełnych danych. Choć dokładność tego podejścia zależy od znajomości rozkładów prawdopodobieństwa cech dla każdej klasy, jego zastosowanie może być bardzo skuteczne, gdy takie informacje są dostępne lub można je dobrze oszacować. Modele bayesowskie są szczególnie przydatne, gdy liczba próbek jest ograniczona, a jednocześnie wymagane jest uzyskanie probabilistycznej interpretacji predykcji.

Zastosowania klasyfikatorów bayesowskich obejmują wiele dziedzin, w których decyzje muszą być podejmowane na podstawie niepewnych lub niepełnych danych, takich jak diagnostyka medyczna, filtrowanie spamu, analiza sentymentu czy rozpoznawanie mowy. Ze względu na swoje solidne podstawy matematyczne, modele te stanowią ważny element teorii statystycznego uczenia maszynowego.

4 Naiwny klasyfikator Bayesa

Naiwny klasyfikator Bayesa to szczególny przypadek klasyfikatora bayesowskiego, który zakłada, że wszystkie cechy są warunkowo niezależne względem siebie przy zadanej klasie. To uproszczenie, choć często nie spełnione w praktyce, umożliwia bardzo efektywne obliczeniowo modelowanie, szczególnie w przypadku danych o dużej liczbie wymiarów, jak teksty czy dane genetyczne.

W praktyce klasyfikator ten szacuje prawdopodobieństwo przynależności do danej klasy, mnożąc aprioryczne prawdopodobieństwo klasy przez iloczyn warunkowych prawdopodobieństw wystąpienia poszczególnych cech. Dzięki temu nie trzeba estymować złożonych wspólnych rozkładów, co znacząco redukuje wymagania względem danych i złożoność obliczeniową.

Pomimo swojego uproszczenia, naiwny Bayes zaskakująco dobrze sprawdza się w wielu rzeczywistych zastosowaniach. Jego skuteczność wynika m.in. z odporności na przeuczenie i szybkości działania, co czyni go popularnym wyborem w takich zadaniach jak klasyfikacja e-maili, analiza treści internetowych, wykrywanie oszustw czy rozpoznawanie autorstwa dokumentów.

5 Rola baggingu w tworzeniu klasyfikatorów

Bagging, czyli Bootstrap Aggregating, to jedna z najprostszych, a zarazem najskuteczniejszych metod zespołowego uczenia maszynowego. Polega ona na tworzeniu wielu modeli uczonych na różnych, losowo próbkowanych zbiorach danych (ze zwracaniem), a następnie agregowaniu ich wyników w celu uzyskania końcowej predykcji.

Ideą stojącą za baggingiem jest redukcja wariancji modelu — w szczególności modeli niestabilnych, takich jak drzewa decyzyjne. Przez uśrednianie wielu niezależnych modeli, bagging zmniejsza wrażliwość na fluktuacje w danych treningowych i zwiększa ogólną dokładność predykcji. W przypadku klasyfikacji, agregacja polega na głosowaniu większościowym, natomiast w przypadku regresji — na uśrednianiu wyników wszystkich modeli.

Bagging jest również fundamentem bardziej zaawansowanych technik, takich jak lasy losowe (Random Forest), które oprócz próbkowania instancji losują także podzbiory cech przy każdym podziale drzewa. Dzięki temu dodatkowo zwiększana jest różnorodność między modelami w zespole, co prowadzi do jeszcze lepszej generalizacji.

Metoda ta znajduje szerokie zastosowanie w praktyce — od wykrywania oszustw, przez klasyfikację obrazów, aż po medycynę i analizę finansową. Jest szczególnie polecana tam, gdzie dostępne są algorytmy bazowe o wysokiej wariancji i konieczne jest zwiększenie stabilności predykcji.

6 Co to jest przeuczenie i jak można mu zapobiegać

Przeuczenie (ang. *overfitting*) to zjawisko, w którym model uczenia maszynowego uczy się danych treningowych zbyt dokładnie, włącznie z przypadkowymi fluktuacjami i szumem. W konsekwencji taki model traci zdolność generalizacji, czyli umiejętność poprawnego przewidywania wyników dla nowych, nieznanych danych. Modele przeuczone uzyskują często bardzo dobre wyniki na zbiorze treningowym, ale ich skuteczność na zbiorze testowym lub w rzeczywistych zastosowaniach znacząco spada.

Zjawisko przeuczenia może wynikać z kilku przyczyn. Najczęstszą jest nadmierna złożoność modelu względem liczby dostępnych danych. Na przykład bardzo głębokie drzewo decyzyjne, sieć neuronowa z wieloma warstwami lub model z dużą liczbą współczynników może skutecznie dopasować się do każdego szczegółu w danych treningowych. Ponadto, zbyt długi czas treningu bez kontroli skutkuje sytuacją, w której model nie tylko uczy się istotnych wzorców, ale również zapamiętuje szum.

Aby przeciwdziałać przeuczeniu, stosuje się różne metody. Jedną z nich jest *regularizacja*, która polega na dodaniu do funkcji kosztu dodatkowego składnika karzącego zbyt duże wartości wag. Przykładami są regularyzacja L1 (Lasso) oraz L2 (Ridge). Kolejną metodą jest *early stopping*, czyli przerwanie treningu w momencie, gdy jakość modelu na zbiorze

walidacyjnym przestaje się poprawiać. Istnieje także podejście polegające na *upraszczaniu modelu*, np. ograniczanie liczby neuronów w warstwach lub ograniczanie głębokości drzewa. Ważną rolę odgrywa też *walidacja krzyżowa*, która pozwala ocenić zdolność generalizacji modelu.

7 Drzewa decyzyjne regresyjne

Drzewa decyzyjne regresyjne to struktury uczenia maszynowego wykorzystywane w problemach regresyjnych, czyli tam, gdzie zmienna wyjściowa przyjmuje wartości ciągłe. Model taki dzieli przestrzeń cech na obszary decyzyjne za pomocą sekwencji reguł logicznych, budując drzewiastą strukturę, w której każdy węzeł decyzyjny dokonuje podziału przestrzeni wejściowej, a liść zawiera przewidywaną wartość odpowiedzi.

Celem przy podziale węzłów w drzewie regresyjnym jest minimalizacja błędu predykcji. Najczęściej stosowaną miarą błędu jest *średni błąd kwadratowy* (ang. Mean Squared Error, MSE), definiowany jako:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Drzewo rozwijane jest rekurencyjnie, aż do spełnienia określonych warunków zatrzymania — mogą to być np. minimalna liczba próbek w węźle, osiągnięcie dopuszczalnego poziomu błędu lub maksymalna głębokość drzewa. Choć bardzo głębokie drzewa mogą uzyskiwać bardzo dobre dopasowanie do danych treningowych, to są one szczególnie podatne na przeuczenie.

Drzewa regresyjne są cenione za prostotę i interpretowalność. Znajdują zastosowanie w takich dziedzinach jak prognozowanie cen nieruchomości, zużycie energii, analiza ryzyka kredytowego, a także wszędzie tam, gdzie istotna jest łatwa interpretacja decyzji modelu.

8 Związek wielkości drzewa klasyfikującego z jego zdolnościami uogólniającymi

Rozmiar drzewa decyzyjnego, mierzony np. jego głębokością, liczbą węzłów lub liści, ma istotny wpływ na zdolność tego modelu do generalizacji. Małe, płytkie drzewa zwykle prowadzą do niedouczenia (ang. *underfitting*), ponieważ nie są w stanie uchwycić złożoności danych. Z kolei drzewa bardzo głębokie mogą przeuczyć się do zbioru treningowego, zapamiętując szczegóły i szum, co skutkuje gorszymi wynikami na nowych danych.

Optymalna wielkość drzewa zależy od charakterystyki zbioru danych. W celu zachowania równowagi między niedouczeniem a przeuczeniem stosuje się różne techniki kontroli złożoności drzewa. Do takich technik należą: ograniczenie maksymalnej głębokości drzewa, minimalna liczba próbek wymagana do rozgałęzienia węzła, ograniczenie liczby liści lub *przycinanie drzewa* (ang. *pruning*) po jego pełnym wytrenowaniu. Przycinanie usuwa te części drzewa, które nie wnoszą istotnej poprawy do jakości predykcji na danych walidacyjnych.

Zrozumienie zależności między wielkością drzewa a jego wydajnością jest kluczowe w kontekście modelowania i interpretacji. Zbyt złożone modele są nie tylko podatne na przeuczenie, ale również trudne do zrozumienia przez człowieka, co bywa problematyczne w zastosowaniach wymagających przejrzystości, takich jak finanse czy medycyna.

9 Miary różnorodności stosowane przy konstrukcji drzew decyzyjnych

Podczas budowy drzew decyzyjnych, węzły są dzielone na podstawie tzw. miar nieczystości lub różnorodności, które określają, jak bardzo dane w danym podzbiorze są mieszane pod względem klas. Celem każdego podziału jest maksymalne zmniejszenie tej nieczystości, co skutkuje bardziej jednorodnymi grupami w liściach drzewa.

Do najczęściej stosowanych miar różnorodności należą: **entropia**, **wskaźnik Giniego** oraz **błąd klasyfikacji**. Entropia, inspirowana teorią informacji, jest zdefiniowana jako:

$$H(p) = - \sum_i p_i \log_2 p_i$$

Im większa entropia, tym większa niepewność co do przynależności klasowej próbek. Wskaźnik Giniego oblicza się jako:

$$G(p) = 1 - \sum_i p_i^2$$

Jest on mniej czuły niż entropia, ale często preferowany ze względu na łatwość obliczeń. Błąd klasyfikacji definiuje się jako $1 - \max p_i$ i jest stosunkowo rzadko używany, ze względu na małą czułość w porównaniu do innych miar.

Podczas budowy drzewa, dla każdego możliwego podziału danych oblicza się zmniejszenie nieczystości — różnicę miary przed i po podziale. Najlepszy podział to ten, który maksymalizuje tę różnicę, co oznacza największy zysk informacyjny. Dobór odpowiedniej miary może wpłynąć na strukturę i skuteczność końcowego modelu.

10 Podstawowe problemy uczenia maszynowego

Uczenie maszynowe to dziedzina oferująca ogromne możliwości, ale również napotykająca liczne wyzwania praktyczne i teoretyczne. Jednym z kluczowych problemów jest odpowiednia reprezentacja danych wejściowych. Dane mogą zawierać szum, błędy pomiarowe, wartości odstające, a także brakujące informacje, co wymaga ich odpowiedniego przetworzenia. Bardzo ważny jest też *dobór cech* — zbyt duża liczba zmiennych może prowadzić do zjawiska przekleństwa wymiarowości, podczas gdy zbyt mała liczba może ograniczyć zdolność modelu do nauki.

Kolejnym problemem jest wybór odpowiedniego algorytmu uczenia. Różne algorytmy mają różne wymagania co do struktury danych, liczby próbek czy odporności na szum. Przeuczenie i niedouczenie to dwa skrajne przypadki, które wymagają od modelarza znalezienia optymalnego punktu równowagi. Ponadto, ważnym aspektem jest ocena jakości modelu — dobór odpowiednich metryk ewaluacyjnych ma kluczowe znaczenie. Dla danych zrównoważonych wystarczy dokładność, ale w przypadku klas nie zrównoważonych warto stosować np. precyzję, czułość czy F1-score.

Nie można także pominąć kwestii *złożoności obliczeniowej*. Niektóre algorytmy, mimo że skuteczne, mogą być bardzo kosztowne czasowo lub pamięciowo. W zastosowaniach rzeczywistych liczy się nie tylko skuteczność, ale też możliwość szybkiego działania i skalowalność rozwiązania. Ostatecznie, rosnącym problemem jest *interpretowalność modeli*. W wielu dziedzinach — takich jak medycyna, prawo czy finanse — konieczne jest, aby decyzje modelu były zrozumiałe i możliwe do uzasadnienia. W takich przypadkach preferowane są modele łatwe do interpretacji, jak drzewa decyzyjne czy regresja liniowa, nawet jeśli ich dokładność nieco ustępuje bardziej złożonym metodom.

11 Podstawowe algorytmy uczenia maszynowego

Uczenie maszynowe obejmuje szeroki wachlarz algorytmów, które można podzielić na kilka głównych kategorii w zależności od typu problemu oraz dostępnych danych. Najważniejszy podział rozróżnia algorytmy uczenia nadzorowanego, nienadzorowanego oraz ze wzmocnieniem.

Uczenie nadzorowane polega na trenowaniu modelu przy użyciu zbioru danych zawierającego zarówno cechy wejściowe, jak i odpowiadające im etykiety. Do najbardziej znanych algorytmów należą regresja liniowa i logistyczna, drzewa decyzyjne, lasy losowe, maszyny wektorów podpierających (SVM), sieci neuronowe oraz algorytmy k-najbliższych sąsiadów (k-NN). Te metody znajdują zastosowanie w klasyfikacji i regresji, czyli przewidywaniu etykiet klasowych lub wartości liczbowych.

W przypadku uczenia nienadzorowanego model uczy się jedynie na podstawie cech wejściowych, bez informacji o docelowych etykietach. Celem może być grupowanie danych (klasteryzacja), odkrywanie struktur (np. analiza skupień), czy też redukcja wymiaru danych. Przykładami są algorytmy k-means, DBSCAN, hierarchiczne metody grupowania, a także PCA (Principal Component Analysis) i autoenkodery.

Uczenie ze wzmocnieniem natomiast opiera się na mechanizmie nagrody. Agent wykonuje akcje w środowisku, ucząc się, które z nich maksymalizują skumulowaną nagrodę. Przykładami są Q-learning, SARSA, czy też metody aktor-krytyk (actor-critic).

Wybór odpowiedniego algorytmu zależy od charakteru danych, dostępnej mocy obliczeniowej, wymagań interpretacyjnych i konkretnych zastosowań. Każdy z algorytmów ma swoje zalety i ograniczenia, dlatego często stosuje się też metody hybrydowe lub zespołowe, które łączą ich mocne strony.

12 Liniowe algorytmy klasyfikacji

Liniowe algorytmy klasyfikacji opierają się na założeniu, że dane można oddzielić liniową granicą decyzyjną, czyli hiperpłaszczyzną w przestrzeni cech. Modele te są jednymi z najprostszych i najbardziej zrozumiałych narzędzi stosowanych w uczeniu maszynowym. Klasycznymi przykładami są regresja logistyczna, perceptron oraz maszyny wektorów podpierających (SVM) w wersji liniowej.

Regresja logistyczna modeluje prawdopodobieństwo przynależności do danej klasy przy pomocy funkcji logistycznej. Jest szczególnie użyteczna w zadaniach klasyfikacji binarnej. Model przyjmuje postać:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x} - b}}$$

Perceptron to prosty liniowy klasyfikator, który uczy się na zasadzie korekcji błędów. Działa efektywnie, o ile dane są liniowo separowalne. Z kolei liniowa wersja SVM dąży do znalezienia hiperpłaszczyzny maksymalnego marginesu oddzielającej klasy.

Liniowe klasyfikatory są efektywne obliczeniowo i dobrze sprawdzają się w problemach, w których dane są wysokowymiarowe, np. przy klasyfikacji tekstu lub danych genetycznych. Ich wadą jest ograniczona zdolność do modelowania skomplikowanych, nieliniowych zależności, co ogranicza ich użyteczność w bardziej złożonych problemach, chyba że zastosuje się metody rozszerzające przestrzeń cech, takie jak funkcje jądra (kernel).

13 Metoda k-najbliższych sąsiadów i jej zastosowania

Metoda k-najbliższych sąsiadów (k-NN) to intuicyjny algorytm, który nie wymaga budowania modelu w fazie treningowej. Klasyfikacja nowej próbki odbywa się poprzez analizę k najbliższych punktów w zbiorze uczącym, zgodnie z wybraną metryką odległości. Do najczęściej stosowanych metryk należą odległość euklidesowa, Manhattan lub Minkowskiego.

Po zidentyfikowaniu k najbliższych sąsiadów, przypisuje się nowemu obiektowi etykietę na podstawie głosowania większościowego w przypadku klasyfikacji, bądź średniej wartości sąsiadów w przypadku regresji. Wybór odpowiedniej liczby sąsiadów (parametru k) jest kluczowy — zbyt małe k może prowadzić do dużej wrażliwości na szum, natomiast zbyt duże może rozmyć granice decyzyjne.

k-NN nie wymaga fazy uczenia i jest odporny na problemy związane z nieprzewidywalnym zachowaniem modelu. Wadą algorytmu jest jego niska wydajność obliczeniowa przy dużych zbiorach danych, ponieważ wymaga on obliczenia odległości do wszystkich punktów uczących w czasie rzeczywistym.

Zastosowania k-NN obejmują m.in. rozpoznawanie obrazów, klasyfikację tekstu, systemy rekomendacyjne oraz detekcję anomalii. Pomimo swojej prostoty, k-NN jest często punktem odniesienia przy ocenie skuteczności bardziej złożonych metod.

14 Logistyczna regresja

Regresja logistyczna to model klasyfikacyjny wykorzystywany w uczeniu nadzorowanym, który służy do przewidywania prawdopodobieństwa przynależności obiektu do określonej klasy. Model ten opiera się na funkcji logistycznej (sigmoidzie), która przekształca dowolną wartość rzeczywistą w przedział $[0, 1]$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{gdzie } z = \mathbf{w}^\top \mathbf{x} + b$$

Trening regresji logistycznej polega na maksymalizacji funkcji wiarygodności lub, równoważnie, minimalizacji funkcji kosztu log-loss. Wagi modelu są optymalizowane zwykle przy użyciu metody gradientu prostego lub jego wariantów, np. SGD czy Adam.

Model ten ma interpretowalne parametry: wagi $\mathbf{w}$ wskazują, jak bardzo dana cecha wpływa na decyzję klasyfikacyjną. Logistyczna regresja dobrze radzi sobie w przypadku dużych, rzadkich zbiorów danych (np. w NLP), jest odporna na przetrenowanie i łatwa do skalowania.

Wadą jest ograniczenie do klasyfikacji liniowej. W bardziej złożonych przypadkach, gdzie granica decyzyjna ma nieliniowy charakter, skuteczność regresji logistycznej może być niższa w porównaniu do bardziej zaawansowanych modeli.

Zastosowania regresji logistycznej obejmują klasyfikację medyczną (np. diagnoza chorób), scoring kredytowy, analizę odejść klientów (churn), oraz predykcję zachowań użytkowników.

15 Kryteria oceny jakości klasyfikatorów

Ocena jakości klasyfikatorów w uczeniu maszynowym jest kluczowa dla wyboru najlepszego modelu do danego zadania. Dobór odpowiednich metryk zależy od charakterystyki problemu, zwłaszcza od równowagi klas w zbiorze danych oraz rodzaju decyzji, jakie model ma wspierać.

Najbardziej podstawową miarą jest *dokładność* (accuracy), określająca procent poprawnych klasyfikacji. Jednak w przypadku niezrównoważonych klas dokładność może być myląca, dlatego stosuje się metryki takie jak *precyzja* (precision), *czułość* (recall) oraz *F1-score*, która jest średnią harmoniczną precyzji i czułości:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Dodatkowo, do analizy wyników klasyfikatorów często wykorzystuje się macierz pomyłek (confusion matrix), która umożliwia szczegółowe śledzenie liczby trafień i pomyłek dla każdej klasy. W przypadku zadań rankingowych lub klasyfikacji probabilistycznej stosuje się miary takie jak AUC-ROC, log-loss, czy Average Precision.

Wybór odpowiednich metryk powinien być dostosowany do kontekstu biznesowego lub badawczego. Przykładowo, w medycynie bardziej istotna może być czułość (wykrycie wszystkich przypadków choroby), natomiast w detekcji oszustw precyzja (minimalizacja fałszywych alarmów) może być ważniejsza. Ostateczna ocena klasyfikatora powinna uwzględniać również jego interpretowalność, czas działania oraz stabilność wyników w różnych warunkach.

16 Lasy losowe

Lasy losowe (ang. Random Forest) to metoda uczenia zespołowego, która polega na połączeniu wielu drzew decyzyjnych w jeden wspólny model predykcyjny. Jest to jedna z najczęściej stosowanych technik w uczeniu nadzorowanym, szczególnie ceniona za swoją skuteczność i odporność na przeuczenie. Kluczowym elementem lasów losowych jest różnorodność modeli bazowych, która pozwala na uzyskanie większej dokładności niż pojedyncze drzewo decyzyjne.

Każde drzewo w lesie budowane jest na podstawie innego podzbioru danych, wybranego metodą bootstrapową, czyli poprzez losowanie ze zwracaniem. Co więcej, przy każdym podziale węzła analizowana jest losowa podgrupa cech, a nie wszystkie dostępne cechy, co dodatkowo zwiększa zróżnicowanie między drzewami. Taka konstrukcja sprawia, że lasy losowe są mniej podatne na przeuczenie, ponieważ błędy poszczególnych drzew mają tendencję do znoszenia się nawzajem.

W klasyfikacji decyzja końcowa pochodzi z głosowania większościowego, natomiast w regresji — z uśrednienia predykcji. Lasy losowe są także w stanie mierzyć istotność cech, co pozwala na lepsze zrozumienie struktury danych. Ich główną wadą może być mniejsza interpretowalność w porównaniu do pojedynczych modeli oraz zwiększona złożoność obliczeniowa.

Zastosowania lasów losowych obejmują bioinformatykę (np. klasyfikacja sekwencji DNA), finanse (predykcja ryzyka), systemy rekomendacyjne, wykrywanie oszustw, analizę sentymentu oraz diagnostykę medyczną.

17 Uczenie bez nadzoru (samoorganizacja)

Uczenie bez nadzoru (ang. Unsupervised Learning) odnosi się do sytuacji, w której algorytm nie posiada informacji o etykietach klas i ma za zadanie samodzielnie odkryć strukturę danych. Tego typu algorytmy są używane do eksploracji danych, wykrywania wzorców i tworzenia reprezentacji danych bez konieczności ich oznaczania.

Jedną z głównych technik jest klasteryzacja, która polega na podziale danych na grupy, w których obiekty są do siebie podobne. Przykładami metod klasteryzacji są: k-means (grupowanie wokół centroidów), DBSCAN (klasteryzacja gęstościowa) oraz metody hierarchiczne. Każda z nich ma inne właściwości — np. k-means dobrze działa przy sferycznych klastrach, a DBSCAN wykrywa klastry o dowolnym kształcie i rozróżnia szum.

Inną istotną grupą technik są metody redukcji wymiaru, takie jak PCA, t-SNE, UMAP oraz autoenkodery. Pozwalają one zmniejszyć liczbę zmiennych opisujących dane przy zachowaniu kluczowych informacji. Dzięki temu możliwa jest efektywna wizualizacja danych w przestrzeniach 2D i 3D.

Uczenie bez nadzoru znajduje szerokie zastosowanie m.in. w segmentacji klientów, eksploracji danych, wykrywaniu anomalii, przetwarzaniu języka naturalnego oraz analizie obrazów. Choć często wyniki wymagają dodatkowej interpretacji, metody te są niezwykle cenne w przypadkach, gdy nie mamy dostępu do danych etykietowanych.

18 Uczenie ze wzmocnieniem (z krytykiem)

Uczenie ze wzmocnieniem (ang. Reinforcement Learning, RL) to podejście inspirowane procesem uczenia się przez próbę i błąd, w którym agent uczy się podejmować optymalne decyzje na podstawie interakcji ze środowiskiem. Każda akcja agenta prowadzi do zmiany stanu środowiska i wygenerowania nagrody, która informuje, czy działanie było korzystne. Celem agenta jest maksymalizacja sumy nagród w dłuższym okresie czasu.

Formalnie, RL opiera się na procesach decyzyjnych Markowa (MDP), które opisują przejścia między stanami w zależności od podjętych akcji. Uczenie może opierać się na funkcji wartości (np. funkcji Q), która ocenia jakość par stan-akcja, albo na bezpośrednim modelowaniu polityki, czyli funkcji przypisującej akcję do stanu.

W architekturze actor-critic, agent składa się z dwóch elementów: aktora, który generuje działania, oraz krytyka, który ocenia skutki tych działań. Krytyk oblicza błąd wartości i dostarcza sygnał do uczenia aktora, co pozwala na szybszą i bardziej stabilną naukę w porównaniu do tradycyjnych metod opartych tylko na Q-learningu.

Uczenie ze wzmocnieniem znalazło zastosowanie m.in. w sterowaniu robotami, grach komputerowych (np. AlphaGo, Dota 2), zarządzaniu energią, dynamicznym przydzielaniu zasobów czy nawet w optymalizacji logistyki i portfeli inwestycyjnych. Główne wyzwania to potrzeba dużej liczby interakcji oraz trudność w osiągnięciu stabilnej konwergencji.

19 Metody optymalizacji w uczeniu maszynowym

Optymalizacja jest sercem procesu uczenia maszynowego. Polega ona na znalezieniu takich parametrów modelu, które minimalizują funkcję kosztu — miarę błędu pomiędzy przewidywaniami modelu a rzeczywistymi danymi. W praktyce oznacza to rozwiązywanie problemów optymalizacji w przestrzeniach często o bardzo dużej liczbie wymiarów.

Najbardziej znaną i powszechnie stosowaną techniką jest spadek gradientu. W klasycznym ujęciu batch gradient descent wykorzystuje cały zbiór danych do obliczenia jednego kroku aktualizacji, co jest kosztowne obliczeniowo. Z kolei stochastyczny spadek gradientu (SGD) dokonuje aktualizacji po każdej próbie, co wprowadza niestabilność, ale znacznie przyspiesza uczenie. Mini-batch gradient descent to kompromis między tymi podejściami.

Nowoczesne optymalizatory, takie jak Adam, RMSProp czy Adagrad, dynamicznie dostosowują tempo uczenia dla każdego parametru osobno. Dzięki temu mogą szybciej zbiegać do minimum funkcji kosztu i radzić sobie z wypłaszczonymi powierzchniami błędu. Wzór aktualizacji w Adamie uwzględnia zarówno średnią gradientów, jak i ich wariancję:

$$\theta \leftarrow \theta - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Optymalizacja napotyka liczne trudności, takie jak obecność minimów lokalnych, punkty siodłowe, wypłaszczony gradient czy eksplodujące gradienty w głębokich sieciach. Wymaga to starannego doboru hiperparametrów, takich jak współczynnik uczenia, rozmiar batcha czy techniki regularyzacji.

20 Metody grupowania danych i ich zastosowanie

Grupowanie danych (klasteryzacja) to jedna z podstawowych technik uczenia nienadzorowanego, której celem jest organizacja danych w klastry, w taki sposób, by obiekty w obrębie jednej grupy były do siebie bardziej podobne niż do obiektów z innych grup. Proces ten pozwala zrozumieć strukturę danych i odkryć ukryte zależności między przykładami.

Najczęściej stosowaną metodą jest k-means, który dzieli dane na k klastrów poprzez iteracyjne przypisywanie punktów do najbliższego centroidu i aktualizację położenia centroidów. Pomimo prostoty, k-means wymaga wcześniejszego ustalenia liczby klastrów i działa najlepiej dla danych o sferycznym kształcie klastrów.

Alternatywą jest DBSCAN, który identyfikuje klastry na podstawie lokalnej gęstości punktów. Nie wymaga on podania liczby klastrów i dobrze radzi sobie z danymi zawierającymi szum. Metody hierarchiczne tworzą z kolei drzewiastą strukturę (dendrogram), umożliwiającą analizę danych na różnych poziomach szczegółowości.

Grupowanie znajduje zastosowanie m.in. w segmentacji klientów, analizie społeczności w sieciach, biologii molekularnej (np. analiza ekspresji genów), analizie dokumentów, obrazów, a także w detekcji oszustw i anomalii w systemach informatycznych.

21 Metody redukcji wymiaru

Redukcja wymiaru (ang. Dimensionality Reduction) to proces przekształcania danych z przestrzeni o wysokim wymiarze do przestrzeni o niższym wymiarze, zachowując jednocześnie możliwie najwięcej informacji. Pozwala to na zmniejszenie złożoności obliczeniowej, poprawę uogólnienia modelu oraz lepszą interpretację danych.

Najbardziej znaną techniką jest PCA (Principal Component Analysis), która polega na wyznaczeniu kierunków (głównych komponentów), wzdłuż których dane mają największą wariancję. Wektory te tworzą nową bazę, do której rzutowane są dane wejściowe. W praktyce wybiera się kilka pierwszych komponentów, które tłumaczą większość zmienności w danych.

W sytuacjach, gdy struktura danych jest nieliniowa, lepsze rezultaty osiąga się za pomocą metod takich jak t-SNE czy UMAP. t-SNE stara się zachować lokalne odległości pomiędzy punktami, co umożliwia uzyskanie zrozumiałych wizualizacji w 2D i 3D. UMAP, będący nowszym algorytmem, działa szybciej i lepiej zachowuje strukturę globalną.

Innym podejściem są autoenkodery, czyli sieci neuronowe uczące się kompresji danych poprzez rekonstrukcję wejścia z ukrytej reprezentacji o niższym wymiarze. Uczą się one nieliniowej transformacji i mogą być stosowane zarówno do redukcji wymiaru, jak i do generowania danych.

Redukcja wymiaru odgrywa kluczową rolę w eksploracji danych, wizualizacji, kompresji, przygotowywaniu danych wejściowych do algorytmów klasyfikacyjnych, a także w detekcji anomalii i selekcji cech.